TELEPÍTÉSI ÚTMUTATÓ

OpenHDID – Ügyfélszolgálati hívóazonosítás

Az OpenHDID konténeres telepítése: követelmények, telepítés lépésről lépésre, a beállítások működése a kész csomagban, hálózat és HTTPS, adatbázis, ütemezett feladatok, naplók, a külső rendszerek bekötése és az átvételi ellenőrzés.

VERZIÓ
1.10.0 · 2026. október 23.

KIADÁS
OpenHDID 1.10.0 · ghcr.io/adamfhun/openhdid

CÉLKÖZÖNSÉG
rendszerüzemeltető, rendszergazda

KAPCSOLÓDÓ
üzemeltetési kézikönyv, felhasználói kézikönyv

TARTALOM

1 Áttekintés
2 Követelmények
3 Telepítés lépésről lépésre
4 A beállítások működése
5 Környezeti változók
6 Hálózat, kiszolgálónév, HTTPS
7 Adatbázis (MariaDB)
8 Háttérfolyamatok és ütemezés
9 Naplók
10 Első beállítás a felületen
11 Külső rendszerek bekötése
12 Ellenőrzés és biztonsági átvétel
13 Kipróbálás demóadatokkal
14 Frissítés új verzióra
15 Használati profilok, magas rendelkezésre állás, Kubernetes
R Rövidítésjegyzék

1 Áttekintés

Az OpenHDID egyetlen konténer-image-ként érkezik (`ghcr.io/adamfhun/openhdid` , `linux/amd64` és `linux/arm64`). Ugyanaz az image minden szerepet ellát, a Compose-fájl a parancs szerint indítja a szolgáltatásokat:

SZOLGÁLTATÁS	PARANCS	FELADAT
`web`	`web`	nginx (mainline, headers-more, TLS 1.3) és PHP-FPM egy konténerben: ügyfélportál, munkatársi panel, gépi API-k. Az nginx és a PHP-FPM Unix-socketen beszél, FastCGI-port nem nyílik.
`worker`	`worker default`	Üzenetsor-feldolgozó: e-mail, SMS, életjel.
`sync-worker`	`worker sync`	A panelről indított ügyféltörzs-átvétel külön soron (legfeljebb 20 perc), hogy ne tartsa fel az üzeneteket.
`scheduler`	`scheduler`	Az ütemező a cron helyett (`php artisan schedule:work`). Pontosan egy példány fusson.
`migrate`	`migrate`	Minden indításkor egyszer lefut: megvárja az adatbázist, lefuttatja a migrációkat és a szerepkör-seedert (amely csak bővít: a szerepkörök kézi módosításait nem állítja vissza), majd kilép. A többi szolgáltatás csak a sikeres lefutása után indul.
`mariadb`	–	Beépített MariaDB 12.3 LTS a `db` profillal. Külső adatbázisnál elhagyható.
`valkey`	–	Valkey (Redis-kompatibilis, BSD-licencű) a `redis` profillal: közös gyorsítótár, munkamenet és üzenetsor több webpéldányhoz.

A telepítési mappa tartalma: `compose.yaml` ; `.env` (alkalmazás); `db.env` (adatbázis-rendszergazda és migrációs fiók); `tls/` (saját tanúsítvány és kulcs); választhatóan `ca/` (belső tanúsítványkiadók). Az image nem tartalmaz beállítást, kulcsot, tanúsítványt és dokumentációt. A dokumentáció a forrásrepó `docs` mappájában és minden kiadás mellékleteként érhető el.

Az image tartalma:

- Alpine Linux 3.24, OpenSSL 3.5 (LTS);
- PHP 8.5 FPM csak a szükséges kiterjesztésekkel: pdo_mysql, intl a teljes ICU-adattal, bcmath, zip, soap, pcntl, redis, OPcache;
- forrásból fordított nginx mainline, csak a használt modulokkal és a beépített headers-more modullal;
- az alkalmazás az éles függőségeivel.

Ami kimaradt: Composer, Node.js, git, fordítóeszközök, PHP-forrás, PEAR, curl-parancs, dokumentáció és tesztek. A konténer a `www-data` felhasználóval (uid 82) fut, csak olvasható gyökér-fájlrendszerrel; írni csak a `storage` kötetre és a memóriában lévő `/tmp` -re tud.

2 Követelmények

ÖSSZETEVŐ	KÖVETELMÉNY	MEGJEGYZÉS
Kiszolgáló	Biztonsági támogatást kapó Linux, x86_64 vagy arm64	rendszeres OS-frissítéssel, szinkronizált órával (NTP): a HMAC-aláírás 300 mp-es időablaka és az SSO-tokenek ellenőrzése is erre épül
Docker	Docker Engine 27+ és Docker Compose 2.24+	a Compose-fájl a beépített `configs.content` és a `depends_on.required` elemet használja; rootless Docker is megfelelő (a 3. fejezet „Rootless Docker” része szerint). **Podman nem támogatott:** a python `podman-compose` a profilhoz kötött `depends_on` miatt `KeyError: 'mariadb'` hibával áll meg, a docker-compose szolgáltató a Podman socketjén nem lett végigvive. Podmanos gépen a Podman eltávolítása és a Docker CE telepítése az út (lent); a közben beállított `DOCKER_HOST` változót és docker contextet is el kell távolítani
Erőforrás	a használati profil szerint (15. fejezet): **5k** egy gép 4 vCPU, 8 GB RAM, 80 GB lemez; **10k** két alkalmazásgép 4 vCPU / 8 GB + külső MariaDB és Valkey; **20k** Kubernetes	kipróbáláshoz 2 vCPU, 4 GB RAM, 20 GB lemez is elég; a profilok méretezését a 15. fejezet és az üzemeltetési kézikönyv 9. fejezete írja le
Név és tanúsítvány	DNS-név, hozzá tanúsítvány teljes lánccal és kulccsal (PEM)	ECDSA P-256 vagy legalább 2048 bites RSA; jelszóval védett kulcs nem használható
Adatbázis	beépített MariaDB 12.3 LTS vagy külső MariaDB/MySQL	utf8mb4; külső kiszolgálóhoz TLS-kapcsolat ajánlott

Docker telepítése. A disztribúció saját Docker-csomagja gyakran régi vagy hiányzik (AlmaLinux, RHEL), ezért a Docker saját tárolója ajánlott. Példa AlmaLinux 9-en (AlmaLinux 10-en a tároló felvétele `dnf config-manager addrepo --from-repofile=…` alakú):

```
sudo dnf remove -y podman podman-docker podman-compose buildah   # ha fent vannak
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
sudo systemctl enable --now docker
sudo usermod -aG docker <üzemeltető felhasználó>
```

A `docker` csoport tagsága csak új bejelentkezéssel él (az `id` parancs mutatja), és root-joggal ér fel: ez a felhasználó csak a telepítés üzemeltetésére szolgáljon. Ellenőrzés: a `docker version` és a `docker compose version` kliens- és szerveroldalt is mutat.

Hálózat.

- Bejövő: 443 a böngészőknek, a telefonközpontnak és a mobil kiszolgálónak; 80 csak a böngészős HTTPS-átirányításhoz kell.
- Kimenő, a konténerekből: Exchange (EWS) vagy SMTP, Ozeki SMS-átjáró, Microsoft Entra és ADFS, az ügyféltörzs gépi felülete, valamint a DNS.
- Az image letöltéséhez a `ghcr.io` kell; zárt hálózatban saját registrybe tükrözhető.
- **Proxy mögött:** a Docker-démon nem örökli a felhasználó `http_proxy` / `https_proxy` változóit, a letöltés 30 másodperc után időtúllépéssel elhal; a proxyt systemd drop-innal kell megadni (üzemeltetési kézikönyv 6. fejezet). Az alkalmazás kimenő kapcsolatai (Exchange, SMS-átjáró, ADFS, Entra, ügyféltörzs) ettől függetlenül a `HDID_HTTP_PROXY` kulccsal mennek proxyn át, a `HDID_NO_PROXY` listában felsorolt belső kiszolgálók kivételével (lásd a „Külső rendszerek” táblázatot). A konténer örökölt `HTTP_PROXY` / `HTTPS_PROXY` változóit az alkalmazás nem használja.

> **Kliens-IP a Docker mögött.**
> Az IP-alapú forgalomkorlát és az auditnapló a kérés valódi forráscímére épül. IPv4-en a Docker megőrzi a kliens címét. IPv6-on, kikapcsolt `ip6tables` vagy a userland-proxy használata mellett viszont a konténer a Docker-átjáró címét látja. Telepítés után a `docker compose logs web` access logjában ellenőrizze, hogy az `ip` mező a valódi kliens címét mutatja-e. Ha nem, kapcsolja be a Docker IPv6-NAT-ját, vagy tegye a webet egy terheléselosztó mögé (`OPENHDID_TLS=off` , 6.5 pont).

---

3 Telepítés lépésről lépésre

A példák a `/opt/openhdid` mappát használják. A parancsokat a `docker` csoport tagja futtatja (2. fejezet); az alkalmazás maga nem root felhasználóként fut.

1. **A telepítési fájlok letöltése** a legfrissebb kiadásból. A kiadások böngészhető oldala a `https://github.com/adamfhun/openhdid/releases/latest` ; a lenti alapcím mindig a legfrissebb kiadás mellékleteire mutat, egy adott verzió a `https://github.com/adamfhun/openhdid/releases/download/v<verzió>` alapcímmel választható. Az alapcím önmagában nem megnyitható oldal (404-et ad), csak a fájlnévvel együtt él. A mappa tulajdonosa a parancsokat futtató fiók (`id -un` ; a `$USER` változót nem minden shell állítja be). A `.env` és a `db.env` titkokat tartalmaz, ezért csak a tulajdonos olvashatja:

```
sudo mkdir -p /opt/openhdid && sudo chown "$(id -un)" /opt/openhdid && cd /opt/openhdid
base=https://github.com/adamfhun/openhdid/releases/latest/download
curl -fsSL -O "$base/compose.yaml"
curl -fsSL -o .env "$base/env.example"
curl -fsSL -o db.env "$base/db.env.example"
curl -fsSL -O "$base/backup.sh"
curl -fsSL --create-dirs -o profiles/5k/compose.yaml "$base/profile-5k.yaml"
chmod 600 .env db.env
chmod 755 backup.sh
```

A `backup.sh` a mentés és a visszaállítás szkriptje (üzemeltetési kézikönyv 7. fejezet); a `profiles/5k/compose.yaml` a használati profil kiegészítő fájlja, amelyre a letöltött `.env` `COMPOSE_FILE` sora mutat (15. fejezet; a 10k profilé a `profile-10k.yaml` melléklet). Enélkül a Compose minden parancsa a `compose file "profiles/5k/compose.yaml" set by COMPOSE_FILE environment variable is invalid` hibával áll meg (`TROUBLESHOOTING.md` 15. fejezet). A profilfájl a kiadás része, frissítésnél a `compose.yaml`-lal együtt cserélődik (14. fejezet).

2. **Kötelező értékek a `.env` fájlban:**
   - `OPENHDID_VERSION` : a letöltött `env.example`-ben már a kiadás pontos verziószáma áll (a `releases/latest` mindig a legfrissebb kiadás fájljait adja), ezt nem kell kikeresni; `latest`-re ne írja át;
   - `APP_URL` : a webhely HTTPS-címe, ebből képződik a kiszolgálónév is;
   - `DB_PASSWORD` : a futási adatbázis-fiók jelszava;
   - `REDIS_PASSWORD` : a Valkey jelszava (a munkamenet, a gyorsítótár és az üzenetsor alapból Valkey-n fut; a 4. lépés parancsa ehhez is ad értéket);
   - `COMPOSE_FILE` : a használati profil kiegészítő fájlja, alapból `compose.yaml:profiles/5k/compose.yaml` (15. fejezet); a profil adja a PHP-FPM, a feldolgozó- és az adatbázis-méreteket;
   - a levelezés, az SMS és az ügyféltörzs beállításai (5. fejezet).

   A `db.env` fájlban a `MARIADB_ROOT_PASSWORD` és a migrációs fiók jelszava (`DB_PASSWORD`) kell. Mindhárom jelszó egyedi, hosszú és véletlenszerű legyen, például `openssl rand -base64 32`. Mindhárom titok fájlból is megadható (`MARIADB_ROOT_PASSWORD_FILE` , `DB_PASSWORD_FILE` a `db.env`-ben és a `.env`-ben), lásd a 4. fejezet „Titkok fájlból” pontját.
3. **Mappák és az image letöltése.** A tanúsítvány (`tls`) és a belső tanúsítványkiadók (`ca`) mappáját a telepítő fiók hozza létre: a Compose a hiányzó csatolt mappát root tulajdonnal hozná létre, és a tanúsítvány másolása a következő lépésben elbukna. Az image a `ghcr.io`-ról jön (zárt hálózatban a 2. fejezet szerint tükrözött registryből); az alkalmazáskulcsot is ez az image készíti, ezért a letöltés már itt kell:

   ```
   mkdir -p tls ca
   docker compose pull
   ```

4. **Alkalmazáskulcs és Valkey-jelszó.** Egyszer kell létrehozni. A parancs két sort ír ki (`APP_KEY=…` és `REDIS_PASSWORD=…`); mindkettőt a `.env` azonos nevű sorába kell írni, az alkalmazáskulcsot a mentésektől elkülönített, védett helyen is meg kell őrizni:

   ```
   docker compose run --rm --no-deps web key
   ```

   Image nélkül ugyanilyen értékeket ad: `echo "base64:$(openssl rand -base64 32)"` és `openssl rand -hex 24`. A kulccsal titkosított adatok (biztonsági válaszok, kódmásolatok, API-tokenek) nélküle nem olvashatók vissza. Meglévő telepítésen soha ne cserélje.
5. **Tanúsítvány.** A teljes lánc és a kulcs a `tls` mappába kerül. A konténer felhasználójának (uid 82) olvashatónak kell lennie:

```
cp /utvonal/fullchain.pem /utvonal/privkey.pem tls/
sudo chown 82:82 tls/fullchain.pem tls/privkey.pem
sudo chmod 444 tls/fullchain.pem && sudo chmod 400 tls/privkey.pem
```

Rootless Dockerrel a `sudo chown 82:82` nem jó: ott a konténer 82-es felhasználója a gazdagépen egy altartományi azonosító (például `165617`), a tulajdonost a konténer névterén belül kell beállítani; a parancsot és a számítást a fejezet végén a „Rootless Docker” rész adja. Sudo nélkül, rootful Dockeren ugyanaz a parancs használható.

6. **Belső tanúsítványkiadók** (választható). Ha az Exchange, az ADFS, az ügyféltörzs vagy a külső adatbázis belső CA-val hitelesít, a CA-tanúsítványt tegye a `ca` mappába (`*.crt` vagy `*.pem`). Indításkor a rendszer tanúsítványtára mellé kerül; a TLS-ellenőrzés kikapcsolására nincs szükség.
7. **Indítás** (az image a 3. lépésben már letöltődött):

   ```
   docker compose up -d
   docker compose ps
   ```

   Az első indításkor sorrendben:

   1. a MariaDB létrehozza az adatbázist és a két fiókot;
   2. a `migrate` lefut, majd kilép (`exited (0)`);
   3. a `web`, `worker`, `sync-worker` és `scheduler` elindul; a `web` állapota egy percen belül `healthy`.
8. **Az első rendszergazda** (jelszót kér, legalább 12 karakter):

   ```
   docker compose exec web php artisan hdid:make-admin admin@example.org --
   name="Rendszergazda"
   ```

   A parancs helyi ügyféltörzsbeli tételt is létrehozza, így a belépési szabály teljesül; a későbbi ügyféltörzs-átvételek ezt a helyi tételt nem tekintik hiányzónak, és ha az e-mail-cím az ügyféltörzsben is megjelenik, a fiókot átkötik rá. A jelszót rejtett prompt kéri; ha a belépés „hibás felhasználónév vagy jelszó” hibát ad, a parancs ugyanarra az e-mailre újrafuttatva új jelszót állít, a valódi elutasítási ok pedig az auditnaplóból kérdezhető le (üzemeltetési kézikönyv 6. fejezet). Ha a `web` még nem fut, a parancs `docker compose run --rm --no-deps web artisan hdid:make-admin …` alakban is megy.
9. **Első ellenőrzés és a telefonközpont kulcsa** (a kulcs egyszer jelenik meg):

   ```
   docker compose exec web php artisan hdid:health
   docker compose exec web php artisan hdid:api-key:create "Prémium telefonközpont"
   callcenter
   ```
10. Belépés a `https://<kiszolgáló>/admin` címen, majd az első beállítás a 10. fejezet szerint, végül az átvételi ellenőrzés a 12. fejezet szerint.

Rootless Docker

A rendszer rootless Dockerrel is fut: a démont nem a root, hanem az üzemeltető felhasználó futtatja, a konténerek felhasználói névtérben élnek, és a tároló a felhasználó saját mappájába kerül. Biztonsági szempontból ez a szigorúbb változat; lemezhely miatt nem érdemes áttérni (lásd a 6. pontot). Ami eltér a

fenti lépésektől:

1. **Telepítés és indítás.** A Docker CE csomagjai mellett a `docker-ce-rootless-extras` csomag és a disztribúció `uidmap`, `slirp4netns` (régebbi kernelnél `fuse-overlayfs`) csomagjai kellenek; a démont az üzemeltető felhasználó állítja be és futtatja:

   ```
   dockerd-rootless-setuptool.sh install
   systemctl --user enable --now docker
   sudo loginctl enable-linger "$(id -un)"
   # a démon kijelentkezés után is fusson
   echo "export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock" >> ~/.bashrc
   # a kliens ehhez a démonhoz beszéljen
   ```

   A `docker` csoporttagság itt nem kell és nem számít (a 2. fejezet erre vonatkozó figyelmeztetései nem érvényesek, mert a démon nem root); a `docker compose` parancsokat ugyanez a felhasználó adja ki, a `docker context ls` mutatja, melyik démonhoz beszél a kliens. Szolgáltatáskezelés: `systemctl --user status|restart docker`, a démon naplója `journalctl --user -u docker`. A kiadás konténerei (`cap_drop: ALL`, `no-new-privileges`, csak olvasható gyökér, `tmpfs`) változtatás nélkül futnak.

2. **Portok.** Az 1024 alatti portot (443, 80) a nem root démon alapból nem kötheti: vagy `sudo sysctl -w net.ipv4.ip_unprivileged_port_start=0` (tartósan egy fájlban a `/etc/sysctl.d/` alatt), vagy magasabb port (`OPENHDID_HTTPS_PORT=8443`, `OPENHDID_HTTP_PORT=8080`) egy előtét vagy tűzfal-átirányítás mögött.

3. **A konténer felhasználója a gazdagépen (subuid, subgid).** A konténer `0`-s (root) felhasználója a gazdagépen az üzemeltető felhasználó; minden más konténerbeli `N` azonosító a `/etc/subuid` és `/etc/subgid` fájlban a felhasználónak kiosztott tartomány `kezdő + N - 1` eleme. Az alkalmazás a `82`-es felhasználóval fut, tehát a gazdagépen a `kezdő + 81` azonosító az övé:

   ```
   grep "^$(id -un):" /etc/subuid /etc/subgid     # például app:165536:65536
   echo $(( 165536 + 82 - 1 ))                     # 165617: ez a konténer 82-
   es felhasználója a gazdagépen
   docker compose exec web id                      # a konténer felől:
   uid=82(www-data)
   ```

   A `sudo chown 82:82` ezért rootless alatt egy idegen fiókhoz rendeli a fájlt, amelyet a konténer nem tud olvasni („Permission denied” a kulcsra, a `web` nem indul). A tartomány `kezdő` értéke gépenként más (gyakori a `100000` vagy a `165536`), mindig a saját sor számít; a `subgid` tartomány ugyanígy értendő a csoportra.

4. **Tanúsítvány és kulcs.** A fájlokat az üzemeltető másolja a `tls` mappába (a konténer felől ekkor a root tulajdona), majd a tulajdonost a konténer névterén belül írja a 82-es felhasználóra; ehhez a `CHOWN` képesség kell, mert a kiadás konténerei minden képességet eldobnak:

   ```
   cp /utvonal/fullchain.pem /utvonal/privkey.pem tls/
   docker compose run --rm --no-deps --user 0 --cap-add CHOWN -v "$PWD/tls":/t
   --entrypoint sh web \
     -c 'chown 82:82 /t/fullchain.pem /t/privkey.pem && chmod 444
   ```

```
/t/fullchain.pem && chmod 400 /t/privkey.pem'
ls -ln tls                                    # a gazdagépen a leképezett
szám (például 165617) a helyes
docker compose run --rm --no-deps web tls-check # futó veremnél: docker
compose exec web openhdid tls-check
```

A `--user 0` itt az üzemeltető saját felhasználója, nem a gazdagép rootja. Tanúsítványcsere után `docker compose exec web openhdid reload`. A `tls` és a `ca` mappa maradjon az üzemeltetőé, 755-tel; a `ca` fájljai 444-gyel olvashatók, tulajdonosváltás nélkül. A `backup.sh` a kulcsot és a `storage` kötetet a konténer saját felhasználójával menti és állítja vissza, így rootless alatt is a konténer által látott tulajdonos áll vissza, root nélkül.

5. **A hívók IP-címe.** A rootless démon alapértelmezett port-továbbítója (`builtin`) a kapcsolatokat a saját címéről adja tovább, így a konténer minden kérést az átjáró címéről lát: az access log `ip` mezője, az IP-alapú forgalomkorlát és az auditnapló címe hamis lenne. Két megoldás: a forrás-IP-t megtartó port-továbbító a démon felhasználói systemd-egységében (`~/.config/systemd/user/docker.service.d/override.conf` , benne `[Service]` és `Environment="DOCKERD_ROOTLESS_ROOTLESSKIT_PORT_DRIVER=slirp4netns"` , majd `systemctl --user daemon-reload && systemctl --user restart docker` ; a Docker dokumentációja szerint újabb kiadásokon a `pasta` hálózati illesztő az `implicit` port-továbbítóval ugyanezt gyorsabban adja), vagy egy TLS-t lezáró előtét a gazdagépen, `OPENHDID_TLS=off` és `TRUSTED_PROXIES` beállítással (6.5). Ellenőrzés: a saját gépéről küldött kérés az access logban a saját címével jelenjen meg, ne a `10.0.2.x` vagy `172.x` átjáróval.
6. **Tároló és lemez.** Az image-ek és a kötetek (az adatbázisé is) a `~/.local/share/docker` alá kerülnek (`docker info --format '{{.DockerRootDir}}'`); ha ez a gyökér-fájlrendszeren van, ugyanaz a lemez telik, mint a `/var/lib/docker` esetében, a hely tehát nem a rootless mód miatt lesz több. Rootful telepítés áttérése: `backup.sh` mentés a régi veremből, annak leállítása, a rootless démon alatt `docker compose up -d` , majd `backup.sh restore` (7. fejezet az üzemeltetési kézikönyvben); a két démon nem látja egymás image-eit és köteteit.

---

4 A beállítások működése a kész csomagban

A beállítások két rétegben élnek.

- **Telepítési értékek:** címek, jelszavak, illesztők, ütemezés, hálózat. Környezeti változók a `.env` fájlban.
- **Üzleti paraméterek:** csomaglisták, azonosítási határok, arculat, adatmegőrzés, belépési módok. A felületen, az *Adminisztráció › Rendszer › Beállítások* alatt szerkeszthetők, az adatbázisban tárolódnak.

Ez a szétválasztás a konténerben is ugyanaz, mint a hagyományos telepítésnél.

1. **Átadás.** A Compose a `.env` minden sorát környezeti változóként adja át a konténereknek (`env_file`). Ugyanebből a fájlból olvassa a saját változóit is (`OPENHDID_VERSION` , portok, profilok). A `db.env` -t csak a `mariadb` és a `migrate` szolgáltatás kapja meg. A konténeren belül nincs `.env`

fájl, a Laravel a folyamat környezetét olvassa.

2. **Biztonságos alapértékek.** Az image-ben beállított alapértékek: `APP_ENV=production`, `APP_DEBUG=false`, `LOG_CHANNEL=stderr`. Amit a `.env` nem ad meg, az ezekkel indul.
3. **Titkok fájlból.** Bármely változó megadható fájlként is: `NÉV_FILE=/run/secrets/…`. Indításkor a `NÉV` változó a fájl tartalmát kapja; ha a kettőt egyszerre adja meg, az hiba. Docker- és Kubernetes-titkokhoz ajánlott, mert a sima környezeti változót a `docker inspect` megmutatja. Ugyanez érvényes a beépített MariaDB fiókjaira: a `db.env`-ben a `MARIADB_ROOT_PASSWORD_FILE` és a migrációs fiók `DB_PASSWORD_FILE` (vagy `DB_USERNAME_FILE`) kulcsa, a `.env`-ben a futási fiók `DB_PASSWORD_FILE` kulcsa használható; az első induláskor a fiókkészítő szkript a fájlok tartalmával hozza létre a fiókokat. A titokfájlt ekkor a `mariadb` és a `migrate` konténerbe is csatolni kell ugyanazon az útvonalon (Compose `secrets` vagy kötet), nem csak az alkalmazás-konténerekbe.
4. **Ellenőrzés induláskor.** Minden szerepkör indulásakor az indítószkript megnézi a kötelező értékeket, és hibás beállítással a konténer érthető hibaüzenettel leáll (`docker compose logs`). Hibának számít:
   - az érvénytelen `APP_KEY`;
   - ha az `APP_URL` nem https;
   - a hiányzó adatbázis-adat;
   - az ismeretlen `APP_ENV` érték: csak `production`, `staging`, `demo`, `local` vagy `testing` lehet, mert például a `prod` vagy a `Production` az éles védelmeket csendben kikapcsolná;
   - élesben (`APP_ENV=production`) az `APP_DEBUG=true`, a mindenkit megbízhatónak vevő `TRUSTED_PROXIES` (`*`, `**`, `0.0.0.0/0`, `::/0`), a `SESSION_SECURE_COOKIE=false`, az üres `DB_PASSWORD` (socketes kapcsolat kivételével), és ha a gyorsítótár, a sor vagy a munkamenet Redis-t használ, az üres `REDIS_PASSWORD`. A konténer ilyenkor 64-es kilépési kóddal áll meg, és a `restart` szabály miatt újraindulgat; az ok a `docker compose logs web` kimenetében. Részletes hibaüzenetekhez az üzemeltetési kézikönyv 5. fejezetének hibakeresési receptje való, nem a `production` környezet debug módja.

   Csak figyelmeztetés jár a naplóba író levélküldésért és SMS-ért, valamint az önaláírt tanúsítványért.
5. **Gyorsítótár.** Az ellenőrzés után a konfiguráció, az útvonalak és az események gyorsítótára a memóriában lévő `/tmp`-be épül. A titkokat is tartalmazó konfigurációs gyorsítótár így lemezre sem kerül. A PHP-FPM dolgozói ezt olvassák, a konténer környezetét nem látják (`clear_env`).
6. **Módosítás:** a `.env` szerkesztése után `docker compose up -d`. A Compose az érintett konténereket újraindítja, a gyorsítótár újraépül; a hagyományos `php artisan optimize` és `queue:restart` itt nem kell. A felületi beállítások azonnal élnek, újraindítás nélkül.

> **APP_KEY**
> Soha nem generálódik magától, és meglévő telepítésen nem cserélhető: cseréje a titkosított adatok, a kulcsolt keresőlenyomatok és az ügyféltörzs tokenjeinek elvesztésével jár. A mentés része legyen, a mentéstől elkülönített hozzáféréssel.

---

5 Környezeti változók

A teljes lista magyar magyarázattal a kiadás `env.example` fájljában található. A legfontosabbak:

Backslash az értékben.

Tartományi felhasználónevet (`DOMAIN\user` , például az ügyféltörzs vagy az NTLM-es Exchange fiókja) idézőjel nélkül vagy aposztrófok között írjon: `EMD_SYNC_USERNAME=DOMAIN\user` vagy `EMD_SYNC_USERNAME='DOMAIN\user'` . Dupla idézőjelben a Compose a `\n` és `\r` sorozatot sortöréssé alakítja, így a `"DOMAIN\nagy"` két sorra esik. Az alkalmazás a nevet JSON-ban, helyesen kódolva küldi, a backslash-t nem kell kettőzni.

Kiadás, hálózat, konténer

KULCS	ALAPÉRTÉK	JELENTÉS
`OPENHDID_VERSION`	–	A futtatott image verziója. A letöltött `env.example` -ben a kiadás száma áll, a kiadási folyamat írja be; élesben mindig pontos verziószám, ne `latest` . Frissítésnél ezt kell átírni (14. fejezet).
`COMPOSE_PROFILES`	`db,redis`	Választható szolgáltatások: `db` beépített MariaDB, `redis` beépített Valkey (alapból mindkettő). Külső adatbázisnál és Valkey-nél (10k profil) üres, a `DB_HOST` és a `REDIS_HOST` a külső kiszolgálóra mutat; a 10k profil `primary` értéke az ütemező és az ügyféltörzs-feldolgozót indítja az elsődleges gépen.
`COMPOSE_FILE`	`compose.yaml:profiles/5k/compose.yaml`	A használati profil kiegészítő fájlja a `compose.yaml` mellett (15. fejezet): `profiles/5k` vagy `profiles/10k` . A profil adja a `HDID_SIZING_PROFILE` értékét, a PHP-FPM folyamatszámokat, a feldolgozók számát és a beépített MariaDB memóriáját; a `.env` azonos nevű sorai felülírják.
`HDID_SIZING_PROFILE`	a profil fájlja adja	A profil neve (`5k` , `10k` , `20k`) a Rendszerállapot „Kapacitás” kártyájának, amely a mért aktív fiókok csúcsát a profil tervezett egyidejű felhasználószámához méri, és 80 %-nál sárgul. Profil nélkül a kártya csak tájékoztat.
`OPENHDID_NFS_SERVER` , `OPENHDID_NFS_PATH`	üres	Csak a 10k profilnál: a két alkalmazásgép közös `storage` kötetének NFS-kiszolgálója és exportált útvonala (15.4 pont).
`OPENHDID_BIND_IP`	`0.0.0.0`	A gazdagép címe, amelyen a 80-as és 443-as port figyel (6.1 pont).
`OPENHDID_HTTPS_PORT` , `OPENHDID_HTTP_PORT`	`443` , `80`	A gazdagép portjai; a konténerben mindig 8443 és 8080.

KULCS	ALAPÉRTÉK	JELENTÉS
`OPENHDID_TLS`	`on`	`on` saját tanúsítvány, `off` terheléselosztó mögött, `selfsigned` csak kipróbáláshoz.
`OPENHDID_TLS_CERT` , `OPENHDID_TLS_KEY`	`/etc/openhdid/tls/…`	A tanúsítvány és a kulcs helye a konténerben.
`OPENHDID_EXTRA_HOSTS`	üres	További kiszolgálónevek az `APP_URL` mellett.
`OPENHDID_HTTP_REDIRECT`	`true`	A 80-as porton a böngészős GET/HEAD 308-cal a HTTPS-címre megy; `false` : minden elutasítva.
`OPENHDID_MONITORING_ALLOW`	üres	A `/health` és `/up` elérésére jogosult címek (IP vagy CIDR).
`OPENHDID_NGINX_WORKERS`	`auto`	nginx-dolgozók száma.
`OPENHDID_FPM_*`	a profil adja (5k: `dynamic` , 16, 4, 4, 8; 10k: `static` , 16; profil nélkül `dynamic` , 8, 2, 2, 4)	PHP-FPM folyamatkezelés és a legtöbb/induló/tartalék folyamat webkonténerenként; egy folyamat 60–80 MB. A képbe épített `php.ini` a `max_execution_time` értékét 300 mp-re állítja (FPM és CLI egyaránt; `request_terminate_timeout` 330 s): az ügyféltörzs-átvétel és a fájlimport 120 mp alatti korláttal el sem indul, és az indításkori ellenőrzés hibával leállítja a konténert, ha egy felülírás ez alá vinné.
`OPENHDID_DB_BUFFER_POOL` , `OPENHDID_DB_MAX_CONNECTIONS`	a profil adja (5k: `2G` , 200; profil nélkül `512M` , 200)	A beépített MariaDB memóriája és kapcsolatszáma; a kapcsolatszám legyen legalább minden PHP-FPM folyamat + a feldolgozók + 20.
`OPENHDID_DEMO_DATA`	`false`	Demóadatok betöltése (13. fejezet); élesben tilos.

Alkalmazás

KULCS	ÉRTÉK ÉLESBEN	JELENTÉS
`APP_ENV` , `APP_DEBUG`	`production` , `false`	Hibaválaszban nincs belső részlet; más kombinációval élesben nem indul.
`APP_KEY`	egyszer generált	Lásd 3. és 4. fejezet.
`APP_URL`	`https://hdid.example.org`	A belépési hivatkozások, az átirányítás és a kiszolgálónév alapja.
`APP_LOCALE` , `APP_TIMEZONE`	`hu` , `Europe/Budapest`	Alapnyelv; megjelenített idők és az ütemező időzónája.

KULCS	ÉRTÉK ÉLESBEN	JELENTÉS
`APP_MAINTENANCE_DRIVER`	`cache`	A karbantartási mód minden példányra hat.
`LOG_CHANNEL`, `LOG_LEVEL`	`stderr`, `warning`	Napló a konténer kimenetére (9. fejezet).
`DB_*`	futási fiók	`DB_HOST=mariadb` a beépített adatbázisnál; a `db.env` a migrációs fiókot adja.
`MYSQL_ATTR_SSL_CA`	üres	Külső adatbázis TLS-kapcsolatához a CA-fájl a konténerben (például `/etc/openhdid/ca/db-ca.pem`).
`SESSION_*`	`redis`, 120, titkosított, `__Host-openhdid_session`, Secure, HttpOnly, Lax	Munkamenet a Valkey-ben; 120 perc inaktivitás. `database` csak kipróbáláshoz, Valkey nélkül.
`CACHE_STORE`, `QUEUE_CONNECTION`	`redis`	Gyorsítótár és üzenetsor a Valkey-ben (`REDIS_HOST=valkey` a beépítettnél, külső kiszolgáló a 10k és 20k profilnál; `REDIS_PASSWORD` élesben kötelező). Ez veszi le a munkamenetek, a zárak és a sor terhét az adatbázisról, és több webpéldányhoz nélkülözhetetlen.
`*_QUEUE_RETRY_AFTER`	1260	Nagyobb legyen az átvétel 1200 mp-es korlátjánál.
`TRUSTED_PROXIES`	`127.0.0.1`	Csak `OPENHDID_TLS=off` mellett: a terheléselosztó címei.
`SANCTUM_TOKEN_EXPIRATION`	43200	A mobilalkalmazás bejelentkezésének lejárata percben.
`HDID_CSP_ENABLED`, `HDID_CSP_EXTRA_SOURCES`	`true`, üres	Tartalombiztonsági szabályzat; csak indokolt, pontos források vehetők fel.
`HDID_PHONE_REGION`	`HU`	A hívószámok értelmezésének alapértelmezett országa.
`HDID_SOURCE_URL`	az image kiadására mutat	A „Forráskód” hivatkozás a portál és a panel láblécében; módosított változatnál a módosított forrásra mutasson (AGPL-3.0, 13. pont).
`SHOW_SC`	`true`	A „Forráskód” hivatkozás megjelenítése; `false` elrejti akkor is, ha a cím meg van adva. Az AGPL-3.0 13. pontja szerint a hálózati felhasználóknak ekkor más úton kell felajánlani a forrást.

Külső rendszerek

KULCSOK	JELENTÉS	
`HDID_HTTP_PROXY`	üres	Kimenő proxy az alkalmazás minden HTTP(S)-kéréséhez (ügyféltörzs, Exchange, Ozeki SMS, ADFS, Entra), a web-, a worker- és az ütemező-konténerből egyaránt. Alakja `http://proxy.example.org:3128`, jelszóval `http://felhasznalo:jelszo@proxy.example.org:3128`; a felhasználónév és a jelszó különleges karakterei %-kódolva (`@` = `%40`, `$` = `%24`, `#` = `%23`). Windows-hitelesítésű (NTLM) proxy nem támogatott. Üresen minden kérés közvetlenül megy. Hibás címmel a konténer nem indul; a használt proxyt (jelszó nélkül) a Rendszerállapot „Kimenő proxy” kártyája mutatja. Az SMTP, az adatbázis és a Valkey nem HTTP, rájuk nem hat; az image letöltésére sem (az a Docker-démon proxyján megy).
`HDID_NO_PROXY`	üres	A proxy nélkül elért kiszolgálók vesszővel: név (az aldomaineket is lefedi, például `belso.example.org` vagy `.belso.example.org`), IP-cím vagy tartomány (`10.0.0.0/8`), `*` = mind. A `localhost`, a `127.0.0.0/8` és a `::1` mindig közvetlen. Ide kerülnek a belső kiszolgálók: tipikusan az ügyféltörzs, az ADFS, a helyi Exchange és az SMS-átjáró.
`MAIL_MAILER`	`ews` / `smtp` / `log`	Levelezési illesztő: `ews` az Exchange webszolgáltatás, `smtp` SMTP-kiszolgáló. A `log` csak naplóba ír, élesben a rendszerállapot figyelmeztet. Próba: `hdid:test-mail cím --now` (11. fejezet).
`MAIL_FROM_ADDRESS`, `MAIL_FROM_NAME`	feladó cím és név	EWS-nél a feladó postafiók legyen az EWS-fiók, vagy annak legyen küldési joga rá.

KULCSOK	JELENTÉS	
`EWS_URL`, `EWS_AUTH`, `EWS_USERNAME`, `EWS_PASSWORD`	`https://…/EWS/Exchange.asmx`; `basic` / `ntlm` / `oauth`	Az Exchange webszolgáltatás címe és hitelesítési módja; `basic` és `ntlm` esetén a postafiók felhasználója és jelszava. Tartományi felhasználónév (`DOMAIN\user`) idézőjel nélkül vagy aposztrófok között írandó. Ha az Exchange csak Windows-hitelesítést kínál (a próbaküldés hibája `HTTP 401 … the server offers: NTLM`), `EWS_AUTH=ntlm` kell: az image 1.8.0-től saját, NTLM-képes libcurl-t hoz (az Alpine csomagja NTLM nélkül épül), amit az indításkori ellenőrzés és a Rendszerállapot Levelezés kártyája is számon kér. Alternatíva a basic engedélyezése az EWS virtuális könyvtáron, vagy küldés SMTP-összekötőn (`MAIL_MAILER=smtp` , `MAIL_HOST` , `MAIL_PORT=587` , STARTTLS).
`EWS_OAUTH_TENANT`, `EWS_OAUTH_CLIENT_ID`, `EWS_OAUTH_CLIENT_SECRET`, `EWS_OAUTH_SCOPE`	Entra-regisztráció adatai; a hatókör alapból `https://outlook.office365.com/.default`	`EWS_AUTH=oauth` esetén: a bérlő GUID-ja, az alkalmazás kliensazonosítója és titka, a postafiókra adott EWS-jogosultsággal. A hatókört csak eltérő felhő esetén kell megadni.
`EWS_VERIFY_TLS`	`true`	Maradjon `true` ; belső tanúsítványkiadóval kiállított Exchange-tanúsítványnál a CA a `ca` mappába kerül.
`MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_SCHEME`	SMTP-kiszolgáló adatai; `smtp` vagy `smtps`	`MAIL_MAILER=smtp` esetén. `smtps` közvetlen TLS (jellemzően 465-ös port), `smtp` vagy üres: STARTTLS-képes kapcsolat (587).
`SMS_DRIVER`	`ozeki` / `log`	SMS-küldés az Ozeki NG átjárón; a `log` csak naplóba ír, élesben a rendszerállapot hibát jelez. Próba: `hdid:test-sms +36… --now` .
`OZEKI_URL`, `OZEKI_USERNAME`, `OZEKI_PASSWORD`	az átjáró API-címe (például `https://sms.example.org:9501/api`), felhasználó, jelszó	A rendszer POST-kérést küld, a hozzáférés nem kerül a címsorba; csak kapcsolódási hibánál ismétel.
`EMD_SYNC_DRIVER`, `EMD_SYNC_API_URL`	`spreadsheet` , teljes export-URL	Átvételi mód: hitelesített POST → CSV/XLSX fájl az export végpontjáról; az export kéréstörzse (JSON) a panel *Beállítások › Ügyféltörzs (EMD)* fülén, csak SuperAdmin szerkeszti. A régi GET JSON API-hoz `json` , `EMD_SYNC_API_TOKEN` (statikus Bearer-token) és `EMD_SYNC_API_TIMEOUT` (60 mp).

KULCSOK	JELENTÉS	
`EMD_SYNC_LOGIN_URL`, `EMD_SYNC_REFRESH_URL`, `EMD_SYNC_USERNAME`, `EMD_SYNC_PASSWORD`	az ügyféltörzs belépési és tokenfrissítési címe, belépési adatai	Belépés: POST JSON `{Password, Username}`; tokenfrissítés: POST JSON `{refreshToken}`. Mindkét válasz `access_token` és `refresh_token`; a `roles` mezőt a rendszer figyelmen kívül hagyja. Sikertelen tokenfrissítés után egyszeri új belépés következik. Tartományi felhasználónév (`DOMAIN\user`) idézőjel nélkül vagy aposztrófok között írandó.
`EMD_SYNC_AUTH_TIMEOUT`, `EMD_SYNC_EXPORT_TIMEOUT`	20 / 600 másodperc	Belépés és fájlletöltés külön időkorláttal; nem a token érvényessége. A teljes háttérfeladat korlátja 1200 mp, a letöltési időkorlát ennél kisebb legyen.
`EMD_SYNC_ID_LIST_URL`	üres	Az ID-lista (például a szervezetek) lekérdezésének címe: POST az exporttal azonos tokennel, a Beállításokban megadott kéréstörzzsel, az `EMD_SYNC_EXPORT_TIMEOUT` időkorlátjával. Csak akkor kell, ha az export kéréstörzse a `{{ ids }}` helyőrzőt használja; üresen az ID-lista ki van kapcsolva. A helyőrző idézőjelek között szövegként (`"ids": "{{ ids }}"` → `"3,12"`), idézőjel nélkül, szögletes zárójelek között JSON-listaként (`"ids": [{{ ids }}]` → `[3,12]`) töltődik ki; az ügyféltörzs hibaválaszának törzse a `docker compose logs worker` kimenetében olvasható.
`EMD_SYNC_EXPORT_FORMAT`, `EMD_SYNC_CSV_DELIMITER`	`xlsx`, `,`	A várt exportfájl `xlsx` vagy `csv`; a CSV-elválasztó egyetlen karakter. A CSV-kódolás a *Beállítások › Ügyféltörzs (EMD)* fülön állítható.
`EMD_SYNC_TOKEN_KEEP_ALIVE`	`true` / `false` (`on` / `off`, `1` / `0` is)	Az ügyféltörzs API-tokenjének 15 percenkénti háttérfrissítése; kikapcsolva az átvétel szükség szerint frissít, sikertelen frissítés után egyszer újra belép. Olvashatatlan érték esetén az alapérték érvényes. Módosítás után `docker compose up -d`.
`EMD_SYNC_QUEUE`	`sync`	A panelről indított átvétel sora; a `sync-worker` szolgáltatás dolgozza fel, így a legfeljebb 20 perces átvétel nem tartja fel az e-maileket és SMS-eket. Ha várakozó átvételt öt percen túl egyetlen feldolgozó sem vesz fel, a rendszerállapot figyelmeztet.

KULCSOK	JELENTÉS	
`EMD_SYNC_CRON`	`0 */3 * * *`	Teljes ügyféltörzs-átvétel 3 óránként; üresen csak kézzel. A 15 percenkénti tokenfrissítés ettől független. A régi `SYNC_*` nevek álnévként működnek; kitöltött új név elsőbbséget kap.
`CALLCENTER_*`	Örökölt, rögzített telefonközponti kulcs és aláírás. Új partnerhez a felületen kezelt API-kulcs ajánlott; a `CALLCENTER_SIGNATURE_TTL` (300) az aláírt kérés időbélyegének megengedett eltérése.	
`USER_*` , `CLIENT_*` (`ADFS` , `ENTRA`)	Négy alkalmazás-regisztráció az egyszeri bejelentkezéshez; az Entra-bérlő csak GUID lehet.	
`USER_ADFS_EMAIL_CLAIM` , `CLIENT_ADFS_EMAIL_CLAIM`	Az ADFS-tokenek igényei (claimek), amelyeket a rendszer sorrendben a fiók e-mail-címével összevet, vesszővel; üresen `emailaddress,email,upn,preferred_username,unique_name` .	
`USER_ADFS_SCOPES` , `CLIENT_ADFS_SCOPES`	Az `openid profile email` mellé kért további hatókörök (például `allatclaims`); nem kötelező, üresen nincs.	
`USER_ADFS_RESOURCE` , `CLIENT_ADFS_RESOURCE`	A Web API azonosítója `resource` paraméterként (ennek a kiadási szabályai kerülnek az access tokenbe); nem kötelező.	
`USER_SSO_MATCH` , `CLIENT_SSO_MATCH`	A céges belépés tokenjéből melyik igény találja meg a fiókot: `email` (alapérték) vagy `employee_id` (a dolgozói azonosító, az ügyféltörzstétel külső azonosítójával összevetve; a token e-mail-címe ekkor nem számít). Ismeretlen érték: a szolgáltató „nincs beállítva”. Módosítás után `docker compose up -d` .	
`USER_SSO_EMPLOYEE_ID_CLAIM` , `CLIENT_SSO_EMPLOYEE_ID_CLAIM`	Dolgozói azonosító módban az igény neve; üresen `employeeId` . ADFS-nél az igénytípus-URI végződése is jó, az ID token után az access token is számít.	
`HDID_SSO_REQUIRE_SUBJECT`	Alapból `true` : a szolgáltató állandó azonosítója (Entra `oid` , ADFS `sub`) nélkül érkező token elutasul. `false` esetén az ilyen belépés kötés nélkül megy, az auditnapló jelzi.	

KULCSOK	JELENTÉS
`CLIENT_LOGIN_REQUIRE_EMD`	Alapból `true`: az ügyfél csak az e-mail-címét viselő, jelen lévő ügyféltörzstétellel léphet be. `false` esetén tétel nélkül, hiányzó tétellel vagy eltérő címmel is beléphet; a lezárás és a csomagjogosultság marad, munkatársra nem hat, a Rendszerállapot sárgán jelzi. Módosítás után `docker compose up -d`.
`HDID_MFA_PROVIDER_AMR`	Kétlépcsős munkatársi belépésnél a céges szolgáltató MFA-ját igazoló további `amr` értékek, vesszővel (beépítve: Entra `mfa`, ADFS `http://schemas.microsoft.com/claims/multipleauthn`); csak bekapcsolt „A céges szolgáltató MFA-jának elfogadása” beállítás mellett számít. ADFS-nél az igényt az `authnmethodsreferences` kiadási szabály adja.
`HDID_ADFS_DEBUG`	`true` esetén minden ADFS-belépés a kérést, a tokenválaszt, a két token fejlécét és tartalmát az érkezett alakban és visszafejtve (személyes adatokkal) a naplóba írja (`docker compose logs web`); az aláírásnak csak a hossza látszik, a frissítő token nem kerül bele. Alapértéke `false`; amíg be van kapcsolva, a „Belépési szolgáltatók” csempe sárga.

6 Hálózat, kiszolgálónév és HTTPS

6.1 Figyelő cím és portok

A konténerben az nginx mindig ugyanazokon a nem privilegizált portokon figyel: 8080 (HTTP) és 8443 (HTTPS), minden konténercímen. Ezekhez nem kell hozzányúlni.

Hogy a gazdagép melyik címén és portján érhető el a szolgáltatás, azt a Compose port-leképezése dönti el a `.env` alapján:

```
OPENHDID_BIND_IP=192.0.2.10    # egyetlen hálózati kártya címe; 0.0.0.0 = minden IPv4-cím
OPENHDID_HTTPS_PORT=443
OPENHDID_HTTP_PORT=80
```

Ha a 80-as portra egyáltalán nincs szükség, a `compose.yaml` `web` szolgáltatásából törölheti a 8080-as leképezést, vagy az `OPENHDID_HTTP_REDIRECT=false` minden HTTP-kérést elutasít. A 443-as porton a tűzfal engedje a klienseket, a telefonközpontot és a mobil kiszolgálót; az adatbázis és a Valkey portja nincs kivezetve a gazdagépre.

6.2 Kiszolgálónév (domain)

- **A kiszolgálónév** az `APP_URL` hostjából képződik; ez az egyetlen hely, ahol meg kell adni. További nevek az `OPENHDID_EXTRA_HOSTS` változóban adhatók meg (a tanúsítványnak ezeket is le kell fednie).
- **Ismeretlen Host fejléc vagy SNI:** a TLS-kézfogás elutasítva, HTTP-n kapcsolatbontás (444). Idegen név alatt az alkalmazás nem szolgálható ki.
- **A 80-as porton** csak a böngészős GET/HEAD kérés kap 308-as átirányítást az `APP_URL` címére. A gépi (`/api/`) és hitelesítési (`/auth/`) útvonalak, illetve minden más metódus kapcsolatbontást kap.
- **Az nginx-konfigurációt** az indítószkript minden induláskor a környezetből állítja elő, csak ellenőrzött értékekkel, majd `nginx -t` próbát futtat. Kézi szerkesztésre nincs szükség.

6.3 HTTPS saját tanúsítvánnyal (TLS 1.3)

Alapértelmezés (`OPENHDID_TLS=on`): a `tls/fullchain.pem` (a kiszolgáló tanúsítványa, utána a köztes tanúsítványok) és a `tls/privkey.pem`. Induláskor a rendszer ellenőrzi:

- a kulcs a tanúsítványhoz tartozik-e;
- a tanúsítvány lefedi-e az `APP_URL` és az extra neveket (a SAN-ban, helyettesítő névvel is);
- a tanúsítvány érvényes-e már, és nem járt-e le; 30 napon belüli lejáratnál figyelmeztet;
- a kulcs típusát és méretét (RSA legalább 2048 bit);
- van-e köztes tanúsítvány a fájlban.

Hibánál a web nem indul, és a napló megmondja az okot. Kézi próba a futó rendszeren:

```
docker compose exec web openhdid tls-check
```

A parancs az `OPENHDID_TLS=on` módhoz tartozik: `selfsigned` és `off` módban nincs mit ellenőriznie, a hiányzó tanúsítványfájl miatt hibával lép ki.

A beállítás kizárólag TLS 1.3-at fogad (`TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_AES_128_GCM_SHA256`). Early data és session ticket nincs. A kulcscsere elsődlegesen az X25519MLKEM768 poszt-kvantum hibrid eljárás, ezt a friss böngészők és az OpenSSL 3.5 támogatja, a régebbi kliensek az X25519-re esnek vissza. Minden HTTPS-válasz HSTS-fejlécet kap (1 év, aldomainekkel). A TLS 1.2 engedélyezése csak dokumentált döntéssel és klienspróbával jöhet szóba, régi partner miatt sem kerülhet vissza a TLS 1.0/1.1.

Tanúsítványcsere: a fájlok cseréje után kapcsolatbontás nélküli újratöltés, amely előtte ellenőriz:

```
docker compose exec web openhdid reload
```

Automatikus megújításnál (például ACME DNS-01 a gazdagépen) ezt a parancsot kösse a megújítás utáni lépéshez. A konténer nem nyit ACME HTTP-01 útvonalat.

6.4 Fejlécek és a kiszolgáló azonosítóinak elrejtése

Az nginx forrásból, csak a szükséges modulokkal készül; a headers-more modul be van fordítva.

- **Server és X-Powered-By:** egyik válaszban sincs, a hibaválaszokban sem (`server_tokens off` , `more_clear_headers` , `expose_php=Off` , `fastcgi_hide_header`).
- **Biztonsági fejlécek minden válaszon egyszer:** HSTS, `nosniff` , `X-Frame-Options DENY` , `Referrer-Policy` , `Permissions-Policy` , `Cross-Origin-Opener-Policy` , CSP és `X-Robots-Tag` .
- **Gyorsítótár:** a dinamikus válaszokon `Cache-Control: no-store` ; a tartalomhash-elt statikus fájlok egy évig gyorsítótárazhatók, előtömörítve.
- **Hibák és tiltások:** saját, termék- és verziómentes hibaoldal; csak az `index.php` futtatható, a pontos nevű fájlok nem érhetők el, a TRACE 405.
- **Access log:** nem tárol URL-t, lekérdezést, Referert, sütit és kéréstörzset.

6.5 Terheléselosztó vagy fordított proxy mögött

`OPENHDID_TLS=off` esetén a konténer csak a 8080-as porton, HTTP-n szolgál ki. A TLS-t ekkor a terheléselosztó zárja le, és ott is TLS 1.3 legyen.

- **`TRUSTED_PROXIES` :** a terheléselosztó címei vagy CIDR-tartománya; a `*` nem használható. Az nginx csak ezektől fogadja el az `X-Forwarded-For` kliens-IP-t; a PHP felé minden kérés HTTPS-ként megy, mert a nyilvános cím HTTPS.
- **A 8080-as port** csak a terheléselosztó felől legyen elérhető (tűzfal vagy belső hálózat).
- **Kulcsok:** a tanúsítvány és a kulcs a terheléselosztón él. Ha a belső szakaszt is titkosítani kell, maradjon `OPENHDID_TLS=on` a konténerben is (újratitkosítás vagy TLS-átengedés).

6.6 Felügyeleti végpontok

A `/health` (az összes ellenőrzés állapota, 200 vagy 503) és a `/up` csak a konténeren belülről (127.0.0.1) és az `OPENHDID_MONITORING_ALLOW` címeiről érhető el, mások 404-et kapnak. A gazdagépről a közzétett porton érkező kérés a Docker-híd átjárócíméről jön (a `docker network inspect openhdid_default` mutatja, például 172.18.0.1), ezért ezt a címet kell a listába venni, különben a gazdagépről is 404 a válasz; a `docker compose exec web php artisan hdid:health` a listától függetlenül működik. A konténer saját egészség-ellenőrzése egy csak a konténeren belül elérhető porton fut, és nem függ ettől a listától.

6.7 Belső nevek feloldása a konténerből

A konténer a Docker beépített DNS-én át a gazdagép DNS-kiszolgálóit kérdezi, ezért a belső DNS-ben szereplő nevet (Exchange, ADFS, ügyféltörzs, SMS-átjáró, külső adatbázis) magától feloldja; nem kell minden nevet felvenni. A gazdagép `/etc/hosts` bejegyzéseit viszont nem látja.

- **A név csak a gazdagép `/etc/hosts` -ában van** (teszt gépen gyakori): `extra_hosts` egy, a `compose.yaml` mellé tett `compose.override.yaml` -ban a `web` , `worker` , `sync-worker` és `scheduler` szolgáltatásnak, majd `docker compose up -d` . A Compose a felülíró fájlt magától összefésüli, a kiadott `compose.yaml` érintetlen marad. Tartósan jobb, ha a név a belső DNS-be kerül.
- **A gazdagép helyi feloldót használ** (a `/etc/resolv.conf` -ban csak `127.0.0.x` áll, például systemd-resolved), vagy a belső DNS csak VPN-en át érhető el: a Dockernek a `/etc/docker/daemon.json` `dns` kulcsában egyszer kell megadni a belső DNS-kiszolgálókat (meglévő fájlnál abba felvéve), majd `sudo systemctl restart docker` és `docker compose up -d --force-recreate` . Így egyetlen nevet sem kell egyenként felvenni.

Egy név vizsgálata (gazdagép kontra konténer), a `compose.override.yaml`-minta és a `daemon.json`-parancsok a kiadás `TROUBLESHOOTING.md` mellékletében.

---

7 Adatbázis (MariaDB)

7.1 Beépített adatbázis

Indítás és kötet. A `COMPOSE_PROFILES=db` a `mariadb:12.3` LTS image-et indítja digesttel rögzítve, a `db` nevű kötettel. A port a gazdagépre nincs kivezetve.

Első induláskor a MariaDB létrehozza:

- a `DB_DATABASE` adatbázist `utf8mb4_unicode_ci` rendezéssel;
- a migrációs fiókot (a `db.env` `DB_USERNAME` / `DB_PASSWORD` értéke), minden joggal az adatbázison;
- a futási fiókot (a `.env` `DB_USERNAME` / `DB_PASSWORD` értéke), csak `SELECT, INSERT, UPDATE, DELETE` joggal.

Ki melyik fiókot használja. A web, a worker és a scheduler csak a futási fiókot látja, sémát nem módosíthat. A migrációk a migrációs fiókkal futnak.

Kiszolgálóbeállítás. Karakterkészlet, `skip-name-resolve`, puffer (`OPENHDID_DB_BUFFER_POOL`, a gép memóriájának 25–50%-a) és kapcsolatszám (`OPENHDID_DB_MAX_CONNECTIONS`).

> **Csak az első induláskor.**
> A fiókokat és az adatbázist a MariaDB az üres kötet első indításakor hozza létre. Jelszócseréhez később a `mariadb` konténerben `ALTER USER` kell, és a `.env` / `db.env` ezzel egyezzen. Fájlból adott titoknál (`*_FILE`) a fájl tartalma számít, ezért jelszócserénél a fájlt és az adatbázis-fiókot együtt kell módosítani.

7.2 Külső adatbázis

A `COMPOSE_PROFILES` üres (vagy csak `redis`). A `DB_HOST` a kiszolgáló neve. Az adatbázist és a két fiókot előre kell létrehozni:

```
CREATE DATABASE openhdid CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'openhdid'@'10.0.0.%' IDENTIFIED BY '…futási jelszó…';
GRANT SELECT, INSERT, UPDATE, DELETE ON openhdid.* TO 'openhdid'@'10.0.0.%';
CREATE USER 'openhdid_migrate'@'10.0.0.%' IDENTIFIED BY '…migrációs jelszó…';
GRANT ALL PRIVILEGES ON openhdid.* TO 'openhdid_migrate'@'10.0.0.%';
```

A hálózaton át a kapcsolat legyen TLS-es: kényszerítse a kiszolgálón (`REQUIRE SSL` vagy `require_secure_transport`), a CA-fájlt tegye a `ca` mappába, és adja meg a `MYSQL_ATTR_SSL_CA=/etc/openhdid/ca/<fájl>` értéket. A `db.env` ekkor is kell a migrációs fiókhoz; a `MARIADB_ROOT_PASSWORD` üresen maradhat.

7.3 Migrációk

- **Mikor fut:** a `migrate` szolgáltatás minden `docker compose up` alkalmával lefut. Ha nincs új migráció, nem változtat semmin.

- **Mit csinál:** legfeljebb 120 másodpercig vár az adatbázisra (`OPENHDID_DB_WAIT_SECONDS`), futtatja a migrációkat, felveszi a hiányzó jogosultságokat és beépített szerepköröket (a kézi módosításokat megőrzi), és a `db.env` migrációs fiókjával dolgozik.
- **Sikertelen migráció:** ha a `migrate` nem 0-val lép ki, a web és a háttérfolyamatok nem indulnak (`docker compose logs migrate`).
- **Csak előre:** a migrációk csak előre futnak; visszalépéshez a frissítés előtti mentés kell.
- **Egyszerre egy:** egyszerre csak egy `migrate` fusson. Valkey mellett a `migrate` a közös gyorsítótár zárja alatt fut (`migrate --isolated`): a 10k profil két alkalmazásgépe egyszerre is indulhat, a második megvárja az elsőt. Kubernetesben egyetlen Job, nem a webpodok indulása.

---

8 Háttérfolyamatok és ütemezés (a cron helyett)

A konténerekben nincs cron-démon, és a gazdagépen sem kell crontab-bejegyzés. Az ütemezett feladatokat a `scheduler` szolgáltatás futtatja (`php artisan schedule:work`): percenként végrehajtja az alkalmazás ütemezését.

Aktiválás: a feladatok maguktól élnek, amíg a `scheduler` fut; ez az alap Compose-telepítés része. Környezeti változóból csak két feladat állítható, a többi időzítése rögzített:

- `EMD_SYNC_CRON` : a teljes átvétel cron-kifejezése; üresen csak kézi átvétel van;
- `EMD_SYNC_TOKEN_KEEP_ALIVE` : a tokenfrissítés be- és kikapcsolása.

Az időpontok az `APP_TIMEZONE` szerint értendők.

GYAKORISÁG	FELADAT
percenként	lejárt azonosítási munkamenetek zárása; ütemező- és sor-életjel; elakadt kimenő üzenetek újrasorolása
15 percenként	`hdid:emd-sync-token` az ügyféltörzs API-tokenjének frissentartása (fájlos illesztő és bekapcsolt `EMD_SYNC_TOKEN_KEEP_ALIVE` mellett)
óránként	lejárt csomag-felülírások zárása; befejezett hívások tisztítása; végesemény nélkül maradt hívások lezárása; a főügyfél-jogosultságukat vesztett ügyfelek kapcsolatainak megszüntetése (védőháló, a csomagváltozás mentésekor azonnal lefut)
naponta 03:30	`hdid:prune` adatmegőrzési tisztítás és az auditnapló archiválása (a beállított kereten túli napok tömörített fájlokba a `storage` kötetre; üzemeltetési kézikönyv 11. fejezet)
`EMD_SYNC_CRON` szerint	`hdid:emd-sync` teljes ügyféltörzs-átvétel (alapból 3 óránként)

Ellenőrzés: `docker compose exec scheduler php artisan schedule:list` , valamint a *Rendszerállapot* oldal „Ütemező” és „Feladatsor” kártyája, amely a percenkénti életjelből dolgozik.

Az ütemezőből mindig pontosan egy példány fusson. A kritikus feladatok a közös gyorsítótár záraival védettek (`withoutOverlapping` , `onOneServer`), így egy véletlen második példány sem futtatja őket kétszer, de ettől még nem kívánatos.

A `worker` és a `sync-worker` :

- óránként szabályosan kilépnek (`--max-time=3600`), és a Compose újraindítja őket; így a memória nem nő korlátlanul;
- minden feladat előtt újraolvassák a felületi beállításokat;
- a panelről indított ügyféltörzs-átvétel csak akkor fut, ha a `sync-worker` fut, és az `EMD_SYNC_QUEUE` értéke `sync` ; különben a rendszerállapot 5 perc várakozás után figyelmeztet.

---

9 Naplók

Minden folyamat a konténer szabványos kimenetére ír, fájlba nem. A konténermotor gyűjti és forgatja a naplókat: a `compose.yaml` `json-file` naplózó-meghajtót használ, szolgáltatásonként legfeljebb 10 × 20 MB-tal (Podman alatt ez a `k8s-file` , ott csak a méretkorlát él). Beállítás nélkül a naplók korlátlanul nőnének; a Docker saját `local` meghajtóját a Podman nem ismeri.

FORRÁS	FORMÁTUM	OLVASÁS
nginx access log	soronként JSON: idő, kliens-IP, kérésazonosító, metódus, státusz, méret, időtartam	`docker compose logs -f web`
nginx hibák, PHP-FPM, alkalmazás	szöveges sorok; az alkalmazás JSON-ra is állítható (`LOG_STDERR_FORMATTER=Monolog\Formatter\JsonFormatter`)	`docker compose logs web`
üzenetsor-feldolgozók	szöveges sorok	`docker compose logs -f worker sync-worker`
ütemező	szöveges sorok	`docker compose logs scheduler`
migráció, adatbázis	szöveges sorok	`docker compose logs migrate mariadb`
auditnapló	adatbázis	felület: *Rendszer › Auditnapló*

Hasznos szűrések:

```
docker compose logs --since 1h web                  # az utolsó óra
docker compose logs --no-log-prefix web | grep '"status":5'   # szerverhibák
docker compose logs --no-log-prefix web | grep -v '^{' # minden, ami nem access log
(hibák, figyelmeztetések)
docker compose logs -f --tail=100 worker            # élő követés
```

Központi naplógyűjtés. Az ASVS V16 elkülönített, módosítás ellen védett tárolást vár. A `compose.yaml` `x-logging` blokkjában a meghajtó cserélhető, például `journald` , `syslog` (TLS-sel), `gelf` vagy `fluentd` ; a Docker-démon szintjén is beállítható. Az access log nem tartalmaz URL-t és

titkot. Az nginx hibasorai és az alkalmazás hibaszövegei viszont tartalmazhatnak URL-t vagy személyes adatot, ezért a továbbításuk korlátozott hozzáféréssel történjen.

Fájlnapló (ha a szervezet ezt kéri): `LOG_STACK=stderr,daily` és `LOG_CHANNEL=stack`. A napi fájlok ekkor a `storage` köteten keletkeznek (`storage/logs`), 30 napos megőrzéssel.

---

10 Első beállítás a felületen

Belépés a `/admin` címen az első rendszergazdával, majd az *Adminisztráció › Rendszer › Beállítások* alatt, ebben a sorrendben:

1. **Csomagok:** a prémium (és ha van, a normál) csomagnevek listája. Csomaglista nélkül egyetlen ügyfél sem tud belépni.
2. **Ügyfélszolgálat:** szintenként e-mail-cím, telefonszám, nyitvatartás.
3. **Arculat és Prémium:** alkalmazásnév, embléma, böngészőikon, színek, háttérképek. Az alapértelmezés semleges színséma; a levelek és a kimutatások is a beállított színeket használják.
4. **Portál:** jogi név, lábléc, adatkezelési és felhasználási hivatkozások.
5. **Ügyfél belépés és SSO:** mely belépési módok élnek. Az SSO bekapcsolása előtt a környezeti változók legyenek kitöltve.
6. **Ügyféltörzs (EMD):** munkatársi és ügyfél-domainlista, oszlopmegfeleltetés, védőkorlátok, CSV-kódolás, az export és az ID-lista kéréstörzse, az ID-lista válaszútvonalai és kijelölése (az ügyféltörzs-átvétel kezelésére jogosult munkatárs).
7. **Azonosítás, Hívások, Adatmegőrzés:** az alapértékek az induláshoz megfelelnek. Az ügyintézői sikertelen ellenőrzések korlátja alapból 50 óránként.

Ezután: *Tartalom › Üzenetsablonok* (próbaküldés), *Rendszer › API-kulcsok*, *Fiókok* (munkatársak, szerepkörök), *Azonosítás › Kérdések*. A felület részletes leírása a felhasználói kézikönyvben található.

---

11 Külső rendszerek bekötése

A konténerben futó parancsok előtagja `docker compose exec web php artisan`.

Ügyféltörzs (Enterprise Master Data, EMD)

1. Töltse ki az `EMD_SYNC_*` értékeket (jelentésük az 5. fejezet táblázatában), majd `docker compose up -d`. Az export és az ID-lista kéréstörzse nem környezeti változó: a panel *Beállítások › Ügyféltörzs (EMD)* fülén áll.
2. A belépés ellenőrzése: `docker compose exec web php artisan hdid:emd-sync-token`.
3. Próbafuttatás: `docker compose exec web php artisan hdid:emd-sync --dry-run`, vagy a panel *Próbafuttatás az API-ból* gombja; utóbbi a `sync-worker`-en fut.
4. Megfelelő eredmény után az éles átvétel.

A védőkorlátok, a domainszűrés és a hibakezelés az üzemeltetési kézikönyv 6. fejezetében olvashatók.

Levelezés

`MAIL_MAILER=ews` az `EWS_*` értékekkel, vagy `smtp` a `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD` és `MAIL_SCHEME` értékekkel. A feladó a `MAIL_FROM_ADDRESS`. Próba az ügyfelek érintése nélkül:

```
docker compose exec web php artisan hdid:test-mail sajat@example.org --now
```

SMS (Ozeki NG)

`SMS_DRIVER=ozeki`, az átjáró HTTPS-címe, felhasználó és jelszó. Próba: `docker compose exec web php artisan hdid:test-sms +36301234567 --now`. Az átjáró felé POST-kérés megy, a hozzáférés nem kerül a címsorba.

Egyszeri bejelentkezés (Entra ID, ADFS)

Négy alkalmazás-regisztráció kell: Entra és ADFS, munkatársaknak és ügyfeleknek külön. A mobilalkalmazás saját nyilvános ügyfélazonosítót kap.

Visszatérési cím: `https://<kiszolgáló>/auth/{user|client}/{entra|adfs}/callback`.

A fiók kötése a címtár-azonosítóhoz:

- az e-mail-cím (vagy a dolgozói azonosító) csak megtalálja a fiókot; az első jogosult céges belépés a fiókot a szolgáltató állandó azonosítójához köti (Entra `oid` a bérlővel, ADFS `sub` a regisztrációval), utána ugyanarra a címre más azonosítóval érkező belépés „más címtár-azonosító” okkal elutasul, a fiók nem zárolódik;
- frissítés után minden meglévő fiók az első belépésekor kötődik; a feloldás rendszergazdai döntés: a lista vagy az adatlap `Belépési kötés törlése` gombja, vagy `docker compose exec web php artisan hdid:sso-unbind email@ceg.hu`;
- az ügyféltörzs szabálya változatlan: a tétel e-mail-címének a fiókéval egyeznie kell; az Entra objektumazonosítója a böngészős és a mobil regisztrációnál ugyanaz;
- ADFS újraregisztrálása (új kliensazonosító) után a következő jogosult belépés a kötést magától megújítja; a titok cseréje nem érinti;
- `HDID_SSO_REQUIRE_SUBJECT=false` csak olyan szolgáltatónál, amely az azonosítót nem adja ki; a Rendszerállapot „Belépési kötések” kártyája a más azonosító miatti elutasításokat mutatja;
- kilépéskor az alkalmazás csak a saját munkamenetét és tokenjét szünteti meg, a Microsoft- vagy ADFS-munkamenetet nem: közös gépen a céges belépés gombja jelszó nélkül visszaléptethet, a védelem a munkaállomás zárolása.

Dolgozói azonosító mint a fiókot megtaláló igény (`USER_SSO_MATCH=employee_id`, ügyfeleknél `CLIENT_SSO_MATCH`): a token dolgozói azonosítója találja meg az ügyféltörzstételt (külső azonosító) és azon át a fiókot; a tétel és a fiók e-mail-egyezése marad, a token e-mail-címe nem számít. Az igény neve a `USER_SSO_EMPLOYEE_ID_CLAIM` (üresen `employeeId`). Kiadatása Entránál igény-leképezési szabályzattal (`user.employeeid` → `employeeId`, a manifestben `acceptMappedClaims: true`), ADFS-nél a Web API kiadási szabályával (`employeeID` → `employeeId`). A helyben felvett tétel (`hdid:make-admin`) ebben a módban nem léphet be; hiányzó igénynél az auditnapló az `employee_id_claim` és `employee_id_problem` mezőben mutatja az okot.

Entra-bérlő:

- a saját címtár GUID-ja kell (domain, `common` vagy `organizations` nem jó);
- a regisztráció legyen egybérlős;
- az `email` igény csak ellenőrzött állapotban számít, különben a felhasználónév (UPN) dönt.

ADFS, a fiókot kötő igény:

- a rendszer a `USER_ADFS_EMAIL_CLAIM` (ügyfeleknél `CLIENT_ADFS_EMAIL_CLAIM`) igényeit sorrendben nézi, az első e-mail-alakú érték dönt; üresen `emailaddress`, `email`, `upn`, `preferred_username`, `unique_name`;
- a név a teljes igénytípus-URI végére is illeszkedik (`…/claims/emailaddress`);
- minden igénynél előbb az ID token, utána az access token számít: a Web API kiadási szabályainak igényei `allatclaims` nélkül csak az access tokenbe kerülnek; az access token akkor számít, ha ugyanazzal a kulccsal aláírt, ettől az ADFS-től származik, nem járt le és ennek az alkalmazásnak szól (`appid`);
- a Web API azonosítója a `USER_ADFS_RESOURCE`, további hatókör a `USER_ADFS_SCOPES` (egyik sem kötelező);
- hogy egy belépés melyik igénnyel sikerült, vagy miért nem, az auditnapló mutatja (az igények nevével, értékük nélkül); a beállítás idejére a `HDID_ADFS_DEBUG=true` a tokenek tartalmát a naplóba írja. Módosítás után `docker compose up -d`.

ADFS beállítása lépésenként. Munkatársaknak és ügyfeleknek külön alkalmazáscsoport kell, ugyanezekkel a lépésekkel (ügyfeleknél a `CLIENT_ADFS_*` kulcsokkal és a `client` visszatérési címmel). Az első négy lépés az ADFS-kiszolgálón, az AD FS Management konzolban történik.

1. **Alkalmazáscsoport.** *Application Groups › Add Application Group*, sablon: „Server application accessing a web API”. Meglévő csoportnál: a csoport tulajdonságai › *Add application…* gomb.
2. **Szerveralkalmazás.** A *Client Identifier* lesz a `USER_ADFS_CLIENT_ID`. Visszatérési cím (*Redirect URI*): `https://hdid.example.org/auth/user/adfs/callback`. A *Configure Application Credentials* lépésben *Generate a shared secret*: ez a `USER_ADFS_CLIENT_SECRET`, és csak itt látszik.
3. **Web API.** Azonosító (*Identifier*): a HDID címe, például `https://hdid.example.org/`. Ugyanez kerül betűre azonosan a `USER_ADFS_RESOURCE` kulcsba (a záró perjel is számít). Az azonosító az ADFS-ben egyedi, ezért az ügyfelek csoportjának Web API-ja eltérőt kap, például `https://hdid.example.org/client/`. Hozzáférési szabályzat (*Access control policy*): a szervezet belépési szabályzata, például „Permit everyone” vagy többlépcsős azonosítást kérő szabályzat. Engedély (*Client Permissions*): a 2. lépés szerveralkalmazása, `openid`, `profile` és `email` hatókörrel (a HDID ezeket kéri).
4. **A cím kiadása.** A Web API *Issuance Transform Rules* fülén *Add Rule › Send LDAP Attributes as Claims*, attribútumtár: Active Directory, `E-Mail-Addresses` → `E-Mail Address`. Az ADFS ezt az igényt `allatclaims` hatókör nélkül csak az access tokenbe teszi, `email` néven.
5. **HDID.** A `.env` kulcsai, utána `docker compose up -d`; végül a *Beállítások* oldal SSO fülén az ADFS-belépés bekapcsolása.

```
USER_ADFS_ISSUER=https://adfs.example.org/adfs
USER_ADFS_CLIENT_ID=<Client Identifier>
USER_ADFS_CLIENT_SECRET=<a megosztott titok>
USER_ADFS_RESOURCE=https://hdid.example.org/
USER_ADFS_EMAIL_CLAIM=email
```

A `USER_ADFS_EMAIL_CLAIM=email` csak az e-mail-igénnyel köti a fiókot: ha az igény hiányzik, a belépés érthető üzenettel elutasul, és a rendszer nem áll át csendben a UPN-re (üresen a lista végén a `upn` is szerepel).

6. **Ellenőrzés.** Egy belépés után az auditnapló `login.succeeded` bejegyzésében `email_claim: email` és `email_token: access_token` áll, a rendszerállapot „Belépési szolgáltatók” kártyája a keresett igényt mutatja. Ha nem így van: az üzemeltetési kézikönyv „Az ADFS-belépés nem találja a fiókot” pontja; a leggyakoribb ok a hiányzó Web API vagy `USER_ADFS_RESOURCE`.

Ellenőrzés PowerShell-lel. Az `ADFS` PowerShell-modul csak az ADFS-kiszolgálókon van meg (RSAT-csomag nincs hozzá, a Web Application Proxyn sincs). Máshonnan távoli munkamenettel futtatható, az ADFS-kiszolgálón helyi rendszergazdai joggal (`adfs01` helyett az ADFS-kiszolgáló neve):

```
Invoke-Command -ComputerName adfs01 -ScriptBlock {
    Get-AdfsServerApplication | Format-List Name, Identifier, ApplicationGroupIdentifier
    Get-AdfsWebApiApplication | Format-List Name, Identifier, ApplicationGroupIdentifier
    Get-AdfsApplicationPermission | Format-List ClientRoleIdentifier,
ServerRoleIdentifier, ScopeNames
}
```

A szerveralkalmazás `Identifier` értéke a kliensazonosító. A HDID csoportjában kell lennie egy Web API-nak, amelynek `Identifier` listája tartalmazza a `USER_ADFS_RESOURCE` értékét, és egy engedélynek, amelyben a `ClientRoleIdentifier` a kliensazonosító, a `ServerRoleIdentifier` a Web API azonosítója.

Telefonközpont és automata menü

Kulcs. A panelen *Rendszer › API-kulcsok*, „Call center / IVR” hatókörrel; vagy parancssorból: `docker compose exec web php artisan hdid:api-key:create "Telefonközpont" callcenter`. A kulcs egyszer jelenik meg.

Küldés. A kulcs az `X-Api-Key` fejlécben küldendő. Az `api_key` lekérdezési paraméter csak örökölt partnernek használható, mert webszerver-naplóba kerülhet; az access log itt nem rögzíti.

Aláírás. Választható HMAC-aláírás: `X-Timestamp` és `X-Signature` fejléc. Az aláírandó szöveg: időbélyeg, metódus, kéréscím a lekérdezéssel, törzs.

Leírás. A teljes gépi felület rendszergazdával belépve a `/docs/ivr` címen olvasható, a mobil kiszolgálóé a `/docs/mobile` címen. A megjelenítőt a böngésző az `unpkg.com` címről tölti be (a tartalombiztonsági szabályzat ezt csak a `/docs/*` oldalakon engedi), ezért a munkaállomásnak internet-elérés kell; zárt hálózaton a `/docs/ivr.json` letölthető, és bármely OpenAPI-nézőben olvasható, ez adható át a telefonközpont szállítójának is.

---

12 Ellenőrzés és biztonsági átvétel

12.1 Működés

- ☐ `docker compose ps`: a `web` `healthy`, a `worker`, `sync-worker`, `scheduler` és `mariadb` fut, a `migrate` `exited (0)`.

- ☐ `docker compose exec web php artisan hdid:health` kilépési kódja 0; a Rendszerállapot oldalon az adatbázis, gyorsítótár, üzenetsor és ütemező zöld (az ütemező az első percben még piros lehet). Ha az ügyféltörzs API-belépése nincs beállítva (csak fájlimport), a „Belépés az ügyféltörzs API-jába” csempe piros marad, és a kód 2.
- ☐ A *Rendszerállapot* fejlécében a futó verzió látszik (a telepített kiadás száma, „Verzió: …”).
- ☐ Próbalevél és próba-SMS megérkezik; a *Kimenő üzenetek* listában „elküldve”.
- ☐ Az ügyféltörzs próbafuttatása hihető számokat ad, az éles átvétel sikeres.
- ☐ Egy ügyfélnek küldött belépési hivatkozás beléptet a portálra; bekapcsolt SSO-nál a belépés végigmegy.
- ☐ Telefonközponti próbahívás megjelenik a vezérlőpulton.
- ☐ A `docker compose logs web` access logjában a kliensek valódi IP-címe látszik (2. fejezet).

12.2 Biztonsági próbák a gazdagépről

```
H=hdid.example.org
curl -sS -D - -o /dev/null https://$H/login          # 200, Cache-Control: no-store
curl -sS -D - -o /dev/null https://$H/favicon.ico    # 200
curl -sS -D - -o /dev/null https://$H/nincs.css      # 404, saját hibaoldal
curl -sS -o /dev/null -w '%{http_code}\n' -X TRACE https://$H/   # 405
curl -sS -o /dev/null -w '%{http_code}\n' https://$H/.env        # 404
curl -sS -D - -o /dev/null http://$H/login           # 308 https://$H/login
openssl s_client -connect $H:443 -servername $H -tls1_3 -verify_return_error -brief
</dev/null   # siker
openssl s_client -connect $H:443 -servername $H -tls1_2 -brief </dev/null
# elutasítva
```

Elvárás: egyetlen válaszban sincs `Server` vagy `X-Powered-By` fejléc; a HSTS és a többi biztonsági fejléc pontosan egyszer szerepel; ismeretlen kiszolgálónévre a kézfogás elutasított. Belső CA esetén a CA-fájlt adja át (`--cacert` , `-CAfile`); a `-k` nem elfogadható bizonyíték.

12.3 OWASP-megfeleltetés

Alap: OWASP ASVS 5.0.0, Top 10:2025, OWASP Docker Security Cheat Sheet. A táblázat a konténeres telepítés védelmeit sorolja fel; teljes ASVS-megfelelőséget nem állít.

TERÜLET	VÉDELEM
A02 hibás konfiguráció, ASVS V13	Nem root (uid 82), csak olvasható gyökér-fájlrendszer, minden Linux-képesség elvéve, `no-new-privileges` , setuid-bitek nélkül. Nincs Server/X-Powered-By fejléc, saját hibaoldal, csak az `index.php` fut. Ismeretlen `APP_ENV` értékkel, hibás éles beállítással (debug, mindenkit megbízhatónak vevő `TRUSTED_PROXIES` , nem biztonságos süti, üres adatbázis- vagy Redis-jelszó) és hibás kimenő proxycímmel (`HDID_HTTP_PROXY`) a konténer nem indul.

TERÜLET	VÉDELEM
A03 ellátási lánc, A08 integritás, ASVS V15	Minden alapimage digesttel, minden letöltés SHA-256-tal rögzítve; az nginx-forrás PGP-aláírással ellenőrzött; lockfájlos build, `npm ci --ignore-scripts`. Kiadás előtt: gitleaks, `composer audit`, `npm audit`, Hadolint, Trivy (javítható HIGH/CRITICAL leletnél a kiadás megáll). Aláírt image (cosign), build provenance és SBOM-igazolás; a CI action-jei commit-SHA-ra rögzítve.
A04 kriptográfia, ASVS V12	Kizárólag TLS 1.3, ML-KEM hibrid kulcscsere, HSTS, indításkori tanúsítvány-ellenőrzés; belső CA a tanúsítványtár mellé, a TLS-ellenőrzés nem kapcsolható ki.
Titkok, ASVS V13.3	Az image-ben nincs titok (gitleaks és Trivy titokkeresés). Titok fájlból (`*_FILE`); konfigurációs gyorsítótár csak memóriában; a migrációs adatbázisfiók jelszava csak a `mariadb` és a `migrate` szolgáltatásnál.
A09 naplózás, ASVS V16	Adatminimalizált JSON access log, forgatás, központi gyűjtés naplózó-meghajtóval, auditnapló az adatbázisban.
Nyitott tételek	A CSP-ben a Livewire/Alpine miatt `unsafe-inline` és `unsafe-eval` van. Egyéb nyitott tételek: az alternatív belépési utak és az MFA (az SSO-szolgáltatónál írható elő), az örökölt IVR URL-es kulcsa és módosító GET-hívásai. Az abszolút munkamenet-korlát, a fiókzárolás és a belépések naplózása a Beállítások › Admin belépés és Ügyfél belépés fülön állítható. Ezek értékelése telepítésenként, felelőssel és határidővel történjen.

13 Kipróbálás demóadatokkal

Tanúsítvány és külső rendszerek nélküli kipróbáláshoz a `.env` értékei:

```
APP_ENV=demo
APP_URL=https://localhost
OPENHDID_TLS=selfsigned
OPENHDID_DEMO_DATA=true
MAIL_MAILER=log
SMS_DRIVER=log
```

Az indítás után a migráció egyszer betölti a demóadatokat:

- ügyintéző: `agent@example.test`, jelszó `password-agent-1`;
- prémium ügyfél: `anna@example.test`, PIN 123456;
- normál ügyfél: `bela@example.test`;
- két hét hívástörténet.

Rendszergazdát a 3. fejezet 8. lépése szerint hozzon létre. A böngésző figyelmeztet az önaláírt tanúsítványra. A kimenő e-mailek és SMS-ek a `docker compose logs worker` kimenetében olvashatók. Demó módban a `hdid:health` kilépési kódja 2 (a `log` illesztők és a bekötetlen ügyféltörzs miatt), a

`tls-check` önaláírt tanúsítványnál nem értelmezhető, a `/health` a gazdagépről a 6.6 pont szerint 404-et ad; a 12.2 pont többi próbája a `-k` kapcsolóval (önaláírt tanúsítvány) elvégezhető.

> **Csak kipróbáláshoz.**
> A demófiókok jelszava nyilvános; `APP_ENV=production` mellett a demóadatok betöltése hibával leáll. Éles telepítéshez új, üres adatbázissal kezdjen.

---

14 Frissítés új verzióra

Minden kiadás mellékleteként jön egy `UPGRADE.md`: ugyanezek a lépések másolható parancsokkal, az adott kiadás verziószámával kitöltve (letöltés ellenőrzőösszeggel, az új telepítési kulcsok listája, visszalépés).

1. Olvassa el a kiadási jegyzetet (GitHub Releases, `CHANGELOG.md`). Választhatóan ellenőrizze az image aláírását (üzemeltetési kézikönyv 8. fejezet).
2. Mentés: `./backup.sh` (adatbázis, `storage` kötet, beállítások, `tls`; üzemeltetési kézikönyv 7. fejezet). Ha a frissítés után vissza kell lépni: `./backup.sh restore`.
3. Az új kiadás fájljai ellenőrzőösszeggel (`SHA256SUMS`): a `compose.yaml`, a `backup.sh` és a használati profil fájlja, amelyre a `.env` `COMPOSE_FILE` sora mutat (az 5k profilnál a `profile-5k.yaml` melléklet a `profiles/5k/compose.yaml` helyre, a 10k profilnál a `profile-10k.yaml` a `profiles/10k/compose.yaml` helyre, mindkét gépen). A `.env`, a `db.env` és a `compose.override.yaml` marad. Az új `env.example`-ből a `.env`-ben még nem szereplő kulcsok átnézése: alapértékük van, csak akkor kell felvenni őket, ha a kiadási jegyzet kéri; az 1.7.0 előtti telepítésen a `COMPOSE_FILE` új kulcs, felvételéhez a profilfájl is kell.
4. Az `OPENHDID_VERSION` átírása az új verzióra, majd:

   ```
   docker compose pull
   docker compose up -d
   docker compose ps
   ```

   A `migrate` lefuttatja az új migrációkat, utána a szolgáltatások az új image-dzsel indulnak újra. Egyetlen webpéldánynál ez néhány másodperces kiesés; kiesés nélküli frissítéshez több példány kell terheléselosztó mögött.
5. Ellenőrzés a 12.1 pont szerint.

A platform frissítése (PHP, nginx, OpenSSL, Alpine) új javítóverzióként jelenik meg, a frissítés menete ugyanez. A kiadott címkék soha nem változnak, ezért egy telepítés mindig pontosan reprodukálható.

---

15 Használati profilok, magas rendelkezésre állás, Kubernetes

A telepítés három használati profil egyikére méretezhető. A profil nem új beállításréteg: a Compose-telepítésnél egy kiegészítő fájl a `compose.yaml` mellett (`profiles/5k`, `profiles/10k`), Kubernetesen egy Kustomize-overlay (`kubernetes/overlays/10k`, `kubernetes/overlays/20k`). A profil adja a PHP-

FPM folyamatszámot, a feldolgozók számát, a beépített adatbázis méretét és a profil nevét (`HDID_SIZING_PROFILE`), amelyet a Rendszerállapot *Kapacitás* kártyája mutat: az utolsó 15 percben aktív fiókok számát, a mai csúcsot és a profil tervezett egyidejű felhasználószámát, 80 % fölött sárgán. Amikor a kártya sárgul, a következő profilra kell lépni.

A profilok feltevései: a portált az ügyfelek 5–8 %-a használja egyszerre (a tervezett érték a 8 %), az ügyintézők száma 20–30, naponta az ügyfelek 1–2 %-a hív. A munkamenet, a gyorsítótár és az üzenetsor mindhárom profilban Valkey-n fut.

15.1 A három profil

	`5K`	`10K`	`20K`
Ügyfél · egyidejű portálhasználó	5 000 · 250–400	10 000 · 500–800	20 000 · 1 000–1 600
Topológia	egy gép, Compose, beépített MariaDB és Valkey	**magas rendelkezésre állás:** két alkalmazásgép Compose-zal terheléselosztó mögött, külső MariaDB (elsődleges + replika) és Valkey, NFS a közös `storage` kötetnek; vagy Kubernetes (`overlays/10k`)	**Kubernetes:** web 3–6 pod (HPA), 3 feldolgozó, 1 ügyféltörzs-feldolgozó, 1 ütemező, migrációs Job; menedzselt MariaDB és Valkey; ReadWriteMany kötet
Erőforrás	4 vCPU, 8 GB RAM, 80 GB lemez	2 × (4 vCPU, 8 GB) alkalmazásgép; MariaDB 4 vCPU, 16 GB; Valkey 1 GB; NFS	node pool 3 × (4 vCPU, 16 GB); webpod 1 vCPU, 1,5–2 GB; MariaDB 8 vCPU, 32 GB; Valkey 2 GB; kötet 50 GB
PHP-FPM	16 folyamat, `dynamic`	2 × 16, `static`	3–6 × 12, `static`
MariaDB	`innodb_buffer_pool_size` 2G, 200 kapcsolat	4–8G, 200	8–16G, 300
Feldolgozók	2 `worker` , 1 `sync-worker` , 1 `scheduler`	1 `worker` gépenként; `sync-worker` és `scheduler` csak az elsődlegesen	3 `worker` , 1 `sync-worker` , 1 `scheduler`
Kiesés hatása	a gép kiesése a szolgáltatás kiesése	egy alkalmazásgép kiesése észrevétlen; adatbázis-átállás a replikára	egy pod vagy node kiesése észrevétlen
Fájl	`profiles/5k/compose.yaml`	`profiles/10k/compose.yaml` mindkét gépen, vagy `kubernetes/overlays/10k`	`kubernetes/overlays/20k`

Számítási alap a saját méretezéshez: PHP-FPM folyamat = kérés/s × válaszidő (s) × 1,5 tartalék; memória = folyamat × 70 MB + 300 MB; `max_connections` ≥ minden PHP-FPM folyamat + a feldolgozók + 20. A válaszidőt a saját környezetben kell mérni (üzemeltetési kézikönyv 9. fejezet).

15.2 5k profil: egy kiszolgáló

Ez az alaptelepítés a 3. fejezet szerint; a letöltött `env.example` már ezt a profilt választja, a profil fájlja a kiadás `profile-5k.yaml` melléklete, amelyet a 3. fejezet 1. lépése a `profiles/5k/compose.yaml` helyre tölt le. A `.env` idevágó sorai:

```
COMPOSE_FILE=compose.yaml:profiles/5k/compose.yaml
COMPOSE_PROFILES=db,redis
SESSION_DRIVER=redis
CACHE_STORE=redis
QUEUE_CONNECTION=redis
REDIS_HOST=valkey
REDIS_PASSWORD=…
```

Ellenőrzés indítás után: `docker compose ps` két `worker` példányt mutat; a Rendszerállapot *Kapacitás* kártyája a `5k profil` nevet és a 400-as tervezett értéket írja; a *Gyorsítótár* kártya `redis`. A profil értékei a `.env` azonos nevű soraival felülírhatók (`OPENHDID_FPM_MAX_CHILDREN`, `OPENHDID_DB_BUFFER_POOL`), ha a gép nagyobb vagy kisebb a tervezettnél.

15.3 Mikor kell lépni

- a *Kapacitás* kártya sárga: a mai csúcs eléri a profil tervezett egyidejű felhasználószámának 80 %-át, vagy az adatbázis-kapcsolatok 80 %-a használatban;
- a *Feladatsor* kártya tartósan sok várakozó üzenetet mutat: előbb egy további `worker` (`docker compose up -d --scale worker=3`), aztán a következő profil;
- a vezérlőpult vagy a portál válaszideje a mérésben (üzemeltetési kézikönyv 9. fejezet) a p95 300 ms fölé megy;
- rendelkezésre állási követelmény: 5 000 ügyfél fölött a kétgépes 10k profil akkor is indokolt, ha a terhelés még nem kívánná.

15.4 10k profil: két alkalmazásgép terheléselosztó mögött

Előfeltételek. Két alkalmazásgép (A: elsődleges, B: másodlagos) azonos Docker-telepítéssel; terheléselosztó, amely a TLS-t lezárja és a két gép 80-as portjára oszt; külső MariaDB magas rendelkezésre állással (elsődleges + replika, egyetlen írható példány; a 7.2 pont két fiókjával); külső Valkey jelszóval és AOF-naplóval (a sor tartósságához); NFS-export a közös `storage` kötetnek, a konténer felhasználója (uid 82) számára írhatóan. Az NFS tartja az arculati képeket, a Livewire ideiglenes feltöltéseit, a szinkronfájlokat és az auditarchívumot: enélkül a két webpéldány nem látja egymás fájljait.

A `.env` mindkét gépen (a 3. fejezet szerint, ezekkel az eltérésekkel; az `APP_KEY` azonos a két gépen; a profil fájlja a kiadás `profile-10k.yaml` melléklete, a `profiles/10k/compose.yaml` helyre töltve: `curl -fsSL --create-dirs -o profiles/10k/compose.yaml "$base/profile-10k.yaml"`):

```
COMPOSE_FILE=compose.yaml:profiles/10k/compose.yaml
COMPOSE_PROFILES=primary          # az A gépen; a B gépen üres
OPENHDID_TLS=off
TRUSTED_PROXIES=10.0.0.5          # a terheléselosztó címe(i)
DB_HOST=db.example.org            # külső MariaDB (írható elsődleges)
REDIS_HOST=valkey.example.org
REDIS_PASSWORD=…
```

```
OPENHDID_NFS_SERVER=nfs.example.org
OPENHDID_NFS_PATH=/export/openhdid-storage
```

A `db.env` a migrációs fiókot adja mindkét gépen; a `mariadb` és a `valkey` szolgáltatás itt nem indul, mert a `db` és a `redis` compose-profil nincs megadva.

1. **A gép:** `docker compose up -d`. A `migrate` a közös Valkey zárja alatt fut, majd elindul a `web`, a `worker`, a `sync-worker` és a `scheduler`.
2. **B gép:** `docker compose up -d`. A `migrate` megvárja az A gép migrációját (zár), és nem változtat semmin; elindul a `web` és a `worker`. Ütemező és ügyféltörzs-feldolgozó itt nem fut: ezekből pontosan egy példány lehet.
3. **Terheléselosztó:** élő-próba a `http://<gép>:80/up` címen (ez csak a PHP indulását mutatja, és nincs forgalomkorlátja); a `/health` a felügyeleti rendszereké, percenként 30 kérés egy címről (`OPENHDID_MONITORING_ALLOW`). A terheléselosztó a `X-Forwarded-For` és `X-Forwarded-Proto` fejlécet adja át; ragadós munkamenet nem kell, a munkamenetek a Valkey-ben közösek.
4. **Ellenőrzés:** mindkét gépen `docker compose ps`; a Rendszerállapot a `10k profil` nevet mutatja, a *Gyorsítótár* és a *Feladatsor* kártya `redis`; az access logban (`docker compose logs web`) az `ip` mező a kliens címe, nem a terheléselosztóé (különben a `TRUSTED_PROXIES` hibás).

Frissítés kiesés nélkül: a B gépen `OPENHDID_VERSION` átírása, `docker compose pull` és `up -d` (a `migrate` itt futtatja az új migrációkat), ellenőrzés, majd ugyanez az A gépen. A két verzió a migráció és a második gép frissítése közti percekben együtt fut; a kiadási jegyzet „Frissítés előtt” sora mondja meg, ha egy kiadásnál ez nem megengedett.

Mentés: a `backup.sh` a beépített adatbázist menti; külső MariaDB-nél az adatbázist a kiszolgáló saját eszközével (`mariadb-dump`, pillanatkép) kell menteni, a `storage` kötetet az NFS-kiszolgálón, a `.env`, `db.env` és `tls` fájlokat mindkét gépen.

15.5 20k profil: Kubernetes

Előfeltételek. Kubernetes-fürt (1.29 vagy újabb) ingress-vezérlővel (a manifeszt az ingress-nginx `proxy-body-size` megjegyzését hordozza; más vezérlőnél a megfelelőjét kell megadni), ReadWriteMany tárolóosztály (NFS, CephFS, Azure Files, EFS), menedzselt vagy operátorral futtatott MariaDB (a 7.2 pont két fiókjával) és Valkey; `kubectl` a kezelőgépen. A manifesztek a kiadás `kubernetes` mappájában vannak: a `base` a közös rész, az `overlays/10k` és `overlays/20k` a méret (replikák, erőforrások, PHP-FPM, a profil neve), az image címkéjét a kiadás írja a `base/kustomization.yaml`-ba.

A `.env` ugyanaz, mint a Compose-telepítésnél (`env.example`), ezekkel az eltérésekkel: `OPENHDID_TLS=off` (a TLS az ingressen zárul le), `TRUSTED_PROXIES` az ingress-vezérlő pod-hálózatának tartománya, `DB_HOST` és `REDIS_HOST` a menedzselt kiszolgálók címe, a `COMPOSE_*` és `OPENHDID_NFS_*` sorok nélkül. A `db.env` a migrációs fiókot adja.

1. **Névtér és titkok:**

```
kubectl create namespace openhdid
kubectl -n openhdid create secret generic openhdid-env --from-env-file=.env
kubectl -n openhdid create secret generic openhdid-db-env --from-env-file=db.env
kubectl -n openhdid create secret tls openhdid-tls --cert=fullchain.pem --
```

```
key=privkey.pem
kubectl -n openhdid create configmap openhdid-ca --from-file=ca/   # belső CA-k,
elhagyható
```

2. **Saját kiszolgálónév:** az `overlays/20k/ingress.yaml` két `hdid.example.org` sorát az `APP_URL` gazdagépére kell írni (vagy saját overlayt készíteni, amely a `20k` -ra épül).
3. **Alkalmazás és migráció:**

   ```
   kubectl apply -k kubernetes/overlays/20k
   kubectl -n openhdid wait --for=condition=complete job/openhdid-migrate --timeout=600s
   kubectl -n openhdid get pods
   ```

   A webpodok a migráció alatt is elindulnak, de a készenléti próbájuk (`openhdid healthcheck`) csak az induló alkalmazással teljesül; a Job a `db.env` migrációs fiókjával fut, a podok a futási fiókkal.
4. **Ellenőrzés:** `kubectl -n openhdid logs deploy/openhdid-web` ; a Rendszerállapot `20k profil` nevet mutat; `kubectl -n openhdid get hpa` a web skálázását.

Frissítés: az `OPENHDID_VERSION` megfelelője itt a `base/kustomization.yaml` `newTag` sora (az új kiadás mappája már az új verziót hordozza). A korábbi migrációs Job törlése (`kubectl -n openhdid delete job openhdid-migrate` , ha a `ttlSecondsAfterFinished` még nem takarította el), majd `kubectl apply -k` és a Job megvárása; a webpodok gördülő frissítéssel cserélődnek (`maxUnavailable: 0`), az ütemező és az ügyféltörzs-feldolgozó `Recreate` stratégiával, egy példányban.

Biztonsági környezet. Minden pod nem root (uid 82), csak olvasható gyökér-fájlrendszerrel, minden Linux-képesség nélkül, `RuntimeDefault` seccomp-profillal; a `/tmp` memóriában lévő `emptyDir` . A PodDisruptionBudget két webpodot tart meg karbantartáskor; a webpodok lehetőleg külön node-okra kerülnek.

Mentés: az adatbázist a menedzselt szolgáltatás eszközével, a `storage` kötetet a tárolóosztály pillanatképével vagy fájlszintű mentéssel; a titkokat (`.env` , `db.env` , tanúsítvány) a fürtön kívül, védett helyen.

15.6 Mérés

A helyes méret mérésből jön. A kiadás nem tartalmaz mérőeszközt; az üzemeltetési kézikönyv 9. fejezete írja le, mit és milyen küszöbbel kell mérni (k6 vagy hasonló eszközzel, a profil forgatókönyveivel), és mikor kell a következő profilra lépni.

Rövidítésjegyzék

RÖVIDÍTÉS	JELENTÉS
EMD	Enterprise Master Data; magyarul ügyféltörzs. A munkatársi és ügyféladatok közös külső törzsadatforrása; a parancsok (`hdid:emd-sync`) és a telepítési kulcsok (`EMD_SYNC_*`) viselik a rövidítést.
ACME	Automatic Certificate Management Environment – automatikus tanúsítványkiadási protokoll.

RÖVIDÍTÉS	JELENTÉS
API	Application Programming Interface – gépi alkalmazási felület.
ASVS	Application Security Verification Standard – tételes alkalmazásbiztonsági ellenőrzési követelmények (OWASP).
CA	Certificate Authority – tanúsítványkiadó.
CSP	Content Security Policy – a böngészőben betölthető és futtatható tartalom szabályzata.
FPM	FastCGI Process Manager – a PHP folyamatkezelője.
GHCR	GitHub Container Registry – a GitHub konténer-image tára.
HSTS	HTTP Strict Transport Security – a böngészőt HTTPS használatára kötelező szabály.
ML-KEM	Module-Lattice Key Encapsulation Mechanism – poszt-kvantum kulcscsere-eljárás (FIPS 203).
OIDC, SSO	OpenID Connect; Single Sign-On – egyszeri, vállalati bejelentkezés.
OWASP	Open Worldwide Application Security Project – nyílt alkalmazásbiztonsági közösség és útmutatók.
SAN	Subject Alternative Name – a tanúsítványban felsorolt nevek.
SBOM	Software Bill of Materials – az image összetevőinek jegyzéke.
SNI	Server Name Indication – a kért kiszolgálónév a TLS-kézfogásban.
TLS	Transport Layer Security – titkosított, hitelesített hálózati kapcsolat.